Software Evolution & Technical Debt

CMP9134: Software Engineering

Dr Francesco Del Duchetto
Lecturer in Robotics and Autonomous Systems

University of Lincoln

24 April 2026

Today's Agenda

Agenda

The Journey So Far

Over the past 11 weeks, we have covered the journey of **building** software:

- **Weeks 1-3:** Agile Planning, Requirements, and UML Models.
- **Weeks 4-6:** System Architecture, Design Patterns, and HCI.
- **Weeks 7-9:** Containerisation (Docker) and Automated CI/CD Pipelines.

You have built your Ground Control Station. Your tests are passing. Your containers are running. Is the Software Engineering process over?

No. In fact, it is just beginning.

Today's Learning Objectives

By the end of this lecture, you should be able to:

1. **Understand** the principles of Software Evolution and Lehman's Laws.
2. **Define** Technical Debt and analyze its impact on software lifecycles.
3. **Identify** common "Code Smells" that indicate poor design.
4. **Apply** Refactoring techniques to improve code quality without altering external behavior.

Agenda

The Reality of Software

Software is not a physical bridge. When a bridge is built, it remains static until it decays. Software, however, is a living entity.

Software Evolution

The process of developing, maintaining, and updating software *after* its initial release to accommodate changing user requirements, hardware updates, or business goals.

Why does software evolve?

- The business environment changes (e.g., new robotics regulations).
- Underlying platforms change (e.g., Python 2 is deprecated for Python 3).
- Users discover new ways to use the system that developers never anticipated.

Lehman's Laws of Software Evolution

In the 1970s, Meir Lehman proposed a set of laws regarding software evolution. They remain universally true today.

1. **The Law of Continuing Change:** A system must continually adapt to its changing environment, or it will become progressively less useful until it dies.
2. **The Law of Increasing Complexity:** As a system evolves, its complexity increases unless explicit work is done to maintain or reduce it.
3. **The Law of Declining Quality:** The quality of a system will appear to decline over time unless it is rigorously maintained and adapted to changes in its operational environment.

Agenda

What is Technical Debt?

As complexity increases (Lehman's 2nd Law), teams often take shortcuts to deliver features faster. This introduces **Technical Debt**.

The Financial Metaphor (Ward Cunningham)

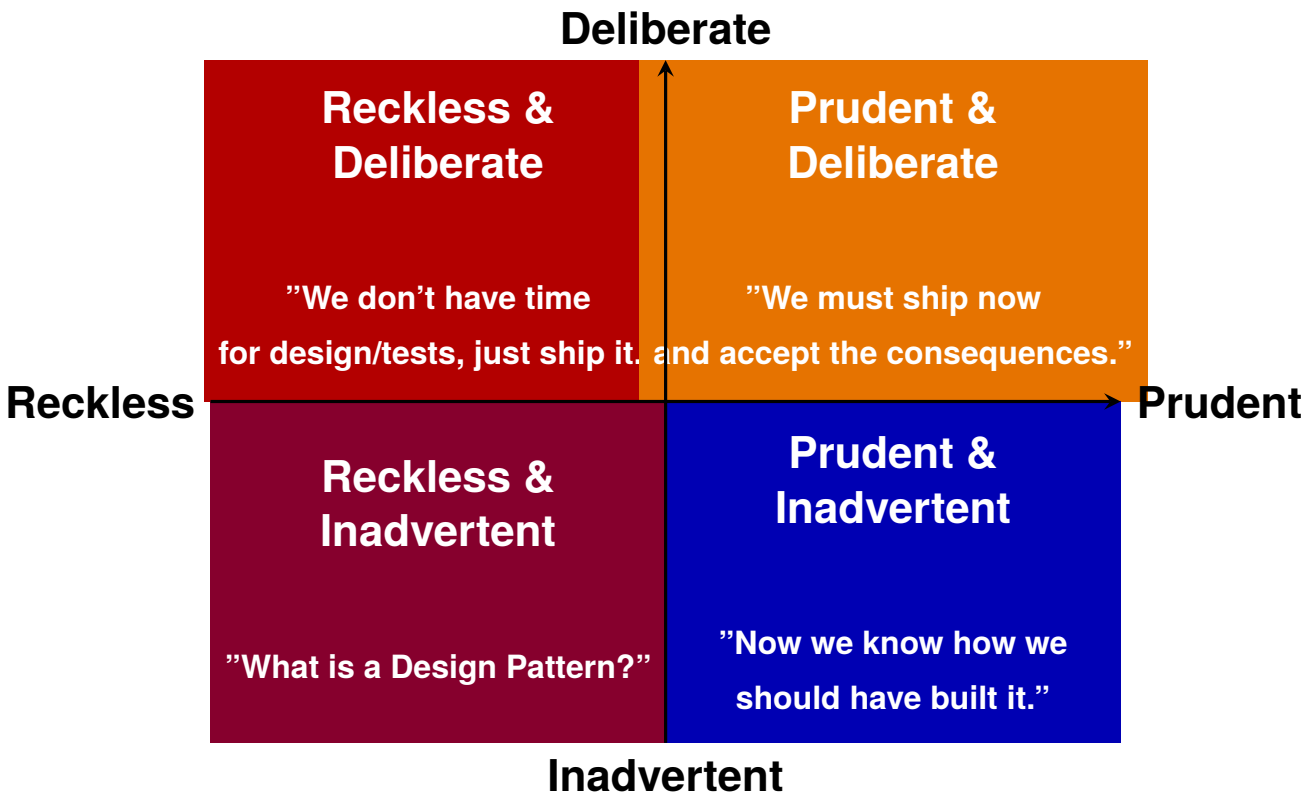
Shipping first-time code is like going into debt. A little debt speeds development so long as it is paid back promptly with a rewrite. The danger occurs when the debt is not repaid. Every minute spent on not-quite-right code counts as **interest** on that debt.

How do we pay the "interest"?

- Slower development times for future features.
- Increased bug rates.
- Developer burnout and frustration.

The Technical Debt Quadrant (Martin Fowler)

Not all technical debt is bad. Sometimes it is a strategic business decision.



Knowledge Check: Classify the Debt

Scenario: Your team is building the Ground Control Station. The deadline is tomorrow. You realize your database queries are unoptimized and might slow down if 1,000 robots connect. You decide to ship the code as-is to meet the deadline, but you immediately create a Jira ticket to rewrite the queries next sprint.

Question: Which quadrant of Technical Debt is this?

1. A) Reckless & Deliberate
2. B) Prudent & Inadvertent
3. C) Prudent & Deliberate
4. D) Reckless & Inadvertent

Knowledge Check: Classify the Debt

Scenario: Your team is building the Ground Control Station. The deadline is tomorrow. You realize your database queries are unoptimized and might slow down if 1,000 robots connect. You decide to ship the code as-is to meet the deadline, but you immediately create a Jira ticket to rewrite the queries next sprint.

Question: Which quadrant of Technical Debt is this?

1. A) Reckless & Deliberate
2. B) Prudent & Inadvertent
3. C) Prudent & Deliberate
4. D) Reckless & Inadvertent

Answer: C (Prudent & Deliberate)

You knew exactly what the consequences were (Deliberate), and you made a calculated business decision while planning for the payback (Prudent).

Agenda

Identifying Debt: Code Smells

How do you know if your codebase has technical debt? You look for **Code Smells**.

A code smell is a surface indication that usually corresponds to a deeper problem in the system. It is not necessarily a bug—the code might work perfectly—but it indicates weaknesses in design that increase the risk of future bugs.

Common Code Smells

- **The God Class:** A single class that controls too much, has too many dependencies, and thousands of lines of code. It violates the Single Responsibility Principle.
- **Long Method:** A function that spans multiple screens. It is doing too many things, making it impossible to unit test effectively.
- **Duplicated Code (Copy-Paste Programming):** The same logic exists in three different places. If a bug is found, you have to remember to fix it in all three places.
- **Shotgun Surgery:** Every time you want to make a simple change (e.g., adding a new robot sensor), you have to make tiny edits to 15 different files.
- **Magic Numbers:** Hardcoded numbers scattered through the code (e.g., `if (status == 4)` instead of `if (status == ROBOT_IDLE)`).

Interactive Task: Spot the Smells

Take 60 seconds. Read the code below. How many distinct "Code Smells" can you find?

```
def do_stuff(x):
    if x == 1:
        # connect to DB
        db.execute("UPDATE robots SET status = 'active' WHERE id = 1")
    elif x == 2:
        # connect to DB
        db.execute("UPDATE robots SET status = 'active' WHERE id = 2")
    # ... repeats 10 more times ...
    print("Done checking. Now let's calculate battery...")
    # ... 50 more lines of battery calculation logic ...
```

Interactive Task: Spot the Smells

Take 60 seconds. Read the code below. How many distinct "Code Smells" can you find?

```
def do_stuff(x):
    if x == 1:
        # connect to DB
        db.execute("UPDATE robots SET status = 'active' WHERE id = 1")
    elif x == 2:
        # connect to DB
        db.execute("UPDATE robots SET status = 'active' WHERE id = 2")
    # ... repeats 10 more times ...
    print("Done checking. Now let's calculate battery...")
    # ... 50 more lines of battery calculation logic ...
```

The Smells:

- **Magic Numbers:** What do `1` and `2` mean? (Should be named constants).
- **Duplicated Code:** The DB connection and update logic is copy-pasted.
- **Long Method / God Class:** Doing DB updates AND battery calculations in one function called `do_stuff`.

Agenda

The Solution: Refactoring

If Technical Debt is the disease, Refactoring is the cure.

Refactoring Definition

A change made to the internal structure of software to make it easier to understand and cheaper to modify **without changing its observable behavior**.

The Golden Rule of Refactoring: You cannot refactor code if you do not have an automated test suite (Week 8)! If you change the internal structure without tests, you are not refactoring; you are just blindly hoping you didn't break anything.

Refactoring Example: Extract Method

The Smell: Long Method with unclear logic.

```
# BEFORE REFACTORING
def process_robot_telemetry(data):
    # Calculate battery percentage
    voltage = data['voltage']
    percentage = (voltage - 10.5) / (12.6 - 10.5) * 100
    if percentage > 100: percentage = 100
    if percentage < 0: percentage = 0

    # Save to database
    db.connect()
    db.execute(f"INSERT INTO telemetry (battery) VALUES ({percentage})")
    db.close()
```

Think-Pair-Share: How do we fix this?

The Bad Code:

```
def process_robot_telemetry(data):
    voltage = data['voltage']
    percentage = (voltage - 10.5) / (12.6 - 10.5) * 100
    if percentage > 100: percentage = 100
    if percentage < 0: percentage = 0

    db.connect()
    db.execute(f"INSERT INTO telemetry (battery) VALUES ({percentage})")
    db.close()
```

Task (2 minutes): Turn to the person next to you. If you had to refactor this using the ”Extract Method” technique, what would your new function signatures look like?

Think-Pair-Share: How do we fix this?

The Bad Code:

```
def process_robot_telemetry(data):
    voltage = data['voltage']
    percentage = (voltage - 10.5) / (12.6 - 10.5) * 100
    if percentage > 100: percentage = 100
    if percentage < 0: percentage = 0

    db.connect()
    db.execute(f"INSERT INTO telemetry (battery) VALUES ({percentage})")
    db.close()
```

Task (2 minutes): Turn to the person next to you. If you had to refactor this using the ”Extract Method” technique, what would your new function signatures look like?

(Let’s look at the solution on the next slide...)

Refactoring Example: Extract Method

The Fix: Extract the logic into smaller, testable, named methods.

```
# AFTER REFACTORING
def process_robot_telemetry(data):
    percentage = calculate_battery_percentage(data['voltage'])
    save_telemetry_to_db(percentage)

def calculate_battery_percentage(voltage):
    percentage = (voltage - 10.5) / (12.6 - 10.5) * 100
    return max(0, min(percentage, 100))

def save_telemetry_to_db(percentage):
    db.connect()
    db.execute("INSERT INTO telemetry (battery) VALUES (?)", (percentage,))
    db.close()
```

Behavior is identical. Readability, security (SQL injection fix), and testability are drastically improved.

When should you refactor?

You don’t need to schedule a ”3-month refactoring sprint”. Refactoring should be a continuous, daily habit.

- **The Rule of Three:** The first time you do something, just do it. The second time, wince at the duplication, but do it. The third time you do the same thing, refactor it.
- **When adding a feature:** If the code is too messy to easily add a new feature, refactor the code first to make the addition easy.
- **The Boy Scout Rule:** ”Always leave the campground cleaner than you found it.” If you open a file to fix a bug and see a poorly named variable, rename it before you commit.

Agenda

This Week's Lab: Paying off your Debt

You have been building your Assessment 1 codebase for several weeks now. It is highly likely you have accumulated Technical Debt.

In the workshop, you will:

- **Task 1: Static Analysis:** Install and run a static analysis tool (like SonarLint, ESLint, or Pylint) on your repository to automatically detect Code Smells.
- **Task 2: Identification:** Identify at least 3 major instances of Technical Debt in your own code.
- **Task 3: Refactoring:** Apply formal refactoring techniques to clean the code, ensuring your automated CI tests still pass afterward!

Reading & Resources

- **Recommended Reading:** Sommerville, *Software Engineering 9th Edition*, Chapter 9 (Software Evolution).
- **Refactoring: Improving the Design of Existing Code** by Martin Fowler (The definitive guide to Code Smells and Refactoring techniques).
- **Refactoring.Guru:** An excellent, visual website detailing dozens of specific code smells and how to fix them.

Q & A

Any Questions?

fdelduchetto@lincoln.ac.uk